Module	Programming Paradigms / COMP1029 (PDP) / Spring Semester
Module Convenor(s)	Doreen Ying Ying Sim

Group O

Index	Student Name	Student ID
1	Tan Ying Qi	20805230
2	WeiTian Tang	20805619
3	Zhuang Yangting	20806311
4	Chen Kequan	20804219
5	Lee chen yi	20805281
6	Wang Sixuan	20804611
7	Daniyal Khan	20806299
8	Aazzu Adam Khalid	20805040

System Architecture and Development Assumptions

1.1. Operational Assumptions

To establish reliability in the unpredictable environment of an actual restaurant our group made three key assumptions:

- **Grid-Based Capacity:** The restaurant was modelled to be a grid where both individuals and robots occupy 1.0 m². This distance was chosen as it aligns with our groups social and physical concerns for the safety of customers and/or staff.
- **Sensor Integration:** For the sensing aspects of the robot our system is limited in that there is no real hardware integration therefore we simulate this in our code using user input. For a complete implementation this data will be fed to the robot by LiDar or Ultrasound sensors.
- **Constant Connectivity:** The robots are co-ordinated using a central system that keeps track of all the current occupancies of each location and the maximum occupancies of those respective locations. This is done to prevent the robots from path finding onto spots that are already filled.

1.2. Proposed Concepts and Dual-Layer Safety Architecture

The Food Serving Multi-Robotic System (FSMS) utilizes a Dual-Layer Safety Architecture as suggested in the coursework specification. Our implementation utilizes these two layers to provide safety by introducing spatial constraints (StaticFoodServing) and constant behavioural requirements (DynamicFoodServing). Details for the implementations of both layers are provided in Section 2 (Handling Static Routing and Capacity) and Section 3 (Handling Dynamic Distancing).

- **Static Zoning (The Macro Layer):** The restaurant is divided into specific zones, (example : "Dining Foyer," "Main Dining Hall") each with their own predefined total area. The system keeps track of occupancy within these zones to ensure that overcrowding never occurs.

- **Dynamic Distancing (The Micro Layer):** Dynamic distancing is the method by which the robots do not collide with one another or customers within a zone below maximum occupancy in the restaurant.

1.3. Extraordinary Features

To achieve the goal of ensuring public health compliance and preventing Robotic failure due to overcrowding we have implemented a few extraordinary features:

- **The Corrective Movement Advisor:** Rather than simply executing an emergency stop when an obstacle is detected, the system features a Movement Advisory Engine. It calculates exact corrective distances (e.g., moving away by 0.5 m or 1.0 m) to proactively resolve spatial conflicts.
- **Intelligent Redirection & Recursive Re-Entry:** The system utilizes a Decision Tree for full-capacity scenarios. It allows the robot to simulate a waiting period or auto-redirect to the next available spot, ensuring the robotic fleet remains productive.
- **Visual Status Transparency:** High-fidelity reporting tools (displaySpotInfo and displayRobotStatus) provide professional-grade summaries of the robot's identity, safety classification, and environmental metrics.

2. Problem I: Handling Static Routing and Capacity

2.1. System Objective

The first major challenge in the simulation was crowd control. To mitigate safety hazards in congested dining areas, the system calculates the maximum safe capacity for specific zones based on a 1.0 m^2 per person distancing rule.

2.2. Location Management (RestrictedSpots.java)

```
FOOD SERVING MULTI-ROBOTIC SYSTEM  (FSMS)
Social Distancing Module  ?  v1.0
COMP1029 Coursework  |  2025-2026

Loaded spot information:

Spot ID      : 1
Name         : Dining Foyer
Total Area   : 50.0 m²
Usable Area  : 35.0 m²
Max Capacity : 35
Avg Wait Time: 5 min

Spot ID      : 2
Name         : Main Dining Hall
Total Area   : 200.0 m²
Usable Area  : 150.0 m²
Max Capacity : 150
Avg Wait Time: 15 min

Spot ID      : 3
Name         : Dining Terrace One
Total Area   : 80.0 m²
Usable Area  : 52.0 m²
Max Capacity : 52
Avg Wait Time: 10 min

Spot ID      : 4
Name         : Dining Terrace Two
Total Area   : 80.0 m²
Usable Area  : 52.0 m²
Max Capacity : 52
Avg Wait Time: 10 min

Spot ID      : 5
Name         : Family Dining Terrace
Total Area   : 120.0 m²
Usable Area  : 84.0 m²
Max Capacity : 84
Avg Wait Time: 12 min

--- Robot Registration ---
Robot ID: 20805230
Robot Name: Tan Ying Qi
```

Picture 1 (Display of RestrictedSpots)

This class acts as the blueprint for the restaurant locations. Because physical fixtures (tables, counters) occupy space, capacity is not calculated using the gross floor area. Instead, the constructor utilizes a usableRatio (a decimal between 0.0 and 1.0) multiplied by the total area to determine the usableArea.

For capacity calculations, the system divides the usable area by a constant DISTANCING_AREA of 1.0 m². The result of this calculation is floored using the Math.floor() function and clamped using Math.max(capacity,1) to ensure there are no possible zero capacity errors. The 5 example spots used in this program and their fields are visible on Picture 1.

2.3. Routing and Occupancy (StaticFoodServing.java)

```
[System] Would you like to perform another action? (yes/no): yes

    RESTAURANT RESTRICTED SPOTS MENU

    [1] Dining Foyer (Max: 35 people, Avg wait: 5 min)
    [2] Main Dining Hall (Max: 150 people, Avg wait: 15 min)
    [3] Dining Terrace One (Max: 52 people, Avg wait: 10 min)
    [4] Dining Terrace Two (Max: 52 people, Avg wait: 10 min)
    [5] Family Dining Terrace (Max: 84 people, Avg wait: 12 min)
    [0] Exit System

Select a spot (1-5, or 0 to exit): 2

[System] Robot 20805230 (Tan Ying Qi) attempting to enter: Main Dining Hall
[System] Current occupancy in 'Main Dining Hall': 62 / 150

?  ENTRY PERMITTED to Main Dining Hall
```

Picture 2 (Under Capacity Scenario)

```
[System] Would you like to perform another action? (yes/no): yes

                RESTAURANT RESTRICTED SPOTS MENU

  [1] Dining Foyer (Max: 35 people, Avg wait: 5 min)
  [2] Main Dining Hall (Max: 150 people, Avg wait: 15 min)
  [3] Dining Terrace One (Max: 52 people, Avg wait: 10 min)
  [4] Dining Terrace Two (Max: 52 people, Avg wait: 10 min)
  [5] Family Dining Terrace (Max: 84 people, Avg wait: 12 min)
  [0] Exit System

Select a spot (1-5, or 0 to exit): 1

[System] Robot 20805230 (Tan Ying Qi) attempting to enter: Dining Foyer
[System] Current occupancy in 'Dining Foyer': 36 / 35

?  ENTRY NOT PERMITTED ? 'Dining Foyer' is at full capacity.
[System] Average wait time for 'Dining Foyer': 5 minute(s).
[System] Would the robot like to wait? (yes/no): yes
[System] Robot is waiting...
[System] Wait time elapsed. Robot is now permitted to enter.


?  ENTRY PERMITTED to Dining Foyer
```

Picture 3 (An Over Capacity Scenario)

This module houses the main execution loop. It initializes an array of five locations with realistic parameters. Current occupancy in our program was simulated using a random number generator. In order to simulate critical situations the aforementioned random number generator was bounded by a value greater than the max capacity for each respective spot (RNG.nextInt(selectedSpot.getMaxCapacity() + 2)). This intentional change allows for extensive testing of our robot rejection and rerouting logic.

The routing logic evaluates the generated occupancy:

- **Under Capacity:** The robots are permitted entry, and the process hands off to the dynamic distancing module. As seen in Picture 2
- **At/Over Capacity:** The entryNotPermitted() method is triggered. The system provides the average wait time and offers the option to hold the position. This is shown in Picture 3. If declined, the user is presented with a menu to select an

alternative spot. If the secondary choice is also at capacity, the robot is returned to standby to prevent traffic bottlenecks.

3. Problem II: Handling Dynamic Distancing

3.1. System Objective

As only monitoring the capacity is insufficient to prevent collisions the system performs real-time dynamic distancing checks upon a robot's entry into a zone to monitor proximity across four directional planes.

3.2. Proximity Control (DynamicFoodServing.java & Robot.java)

```
[System] Proceeding to dynamic distancing check...

Enter distance from LEFT (m): 0.3
Enter distance from RIGHT (m): 1
Enter distance from FRONT (m): 1
Enter distance from BACK (m): 0.3
Warning: close distance detected.
Move away from LEFT by 1.0 m

Move away from BACK by 1.0 m
```

Picture 4 (A Casual Contact Scenario)

The Robot class encapsulates the robot's identity and state, utilizing private instance variables (robotId, robotName, contactStatus) managed via getter and setter methods. Upon initialization, the contact status defaults to "Safe".

The DynamicFoodServing class manages the proximity logic against a constant SAFE_DISTANCE of 1.0 m. The checkDynamicDistance() method processes user input for left, right, front, and back distances. In a complete implementation this method would read input from a sensor. If any single directional value falls below the 1.0 m threshold,

the system updates the robot's status to "Casual Contact", issues a warning, and initiates the giveMovementAdvice() method.

The movement advisor applies conditional logic via getMoveDistance(): if the detected distance is less than 0.5 m, the system commands a 1.0 m retreat; otherwise, it commands a 0.5 m retreat. Finally, displayRobotStatus() outputs the updated condition, ensuring full transparency of the dynamic distancing check.

4. Object-Oriented Programming (OOP) Implementation

The system's architecture relies heavily on core OOP paradigms to maintain a modular and scalable codebase:

- **Classes and Objects:** System responsibilities are divided among the Robot, DynamicFoodServing, RestrictedSpots and StaticFoodServing classes. Object instantiation is used practically, such as creating RestrictedSpots arrays to manage dining areas and passing Robot instances through routing methods.
- **Encapsulation:** Data integrity is maintained, notably within the Robot and RestrictedSpots classes, by declaring variables as private and controlling access exclusively through getter and setter methods.
- **Inheritance:** The codebase explicitly demonstrates inheritance (public class StaticFoodServing extends DynamicFoodServing). This allows the static entry-control module to inherit and reuse the dynamic distancing behaviors defined in the parent class without code duplication. While the system prioritizes robust class design and inheritance over advanced polymorphic method overriding, the implemented structures thoroughly satisfy the requirements for a logical, object-oriented application.

5. Development Experiences and Challenges

In the creation of this system we have gained good practical experience in Java object-oriented programming. During the initial development phase, we faced several problems which include issues with:

- Calculating the true max capacity
- Integrating the Dynamic and Static Food Serving modules
- Treatment of boundary cases (most notably the one where the capacity is 0)

These issues were only resolved after extensive research and thorough co-operation between group mates.

6. Conclusion and Discussion

Through the development of the Food Serving Multi-Robotic System (FSMS), our group has shown how basic Java programming concepts can help solve important safety and operation problems in the Malaysian hospitality sector. By using a Dual-Layer Safety Architecture, the system effectively handles both large-scale spatial constraints and small-scale behavior needs.

Importantly, the system focuses on the comfort of diners, which is a main goal of this simulation. At the macro level, a grid-based capacity of 1.0 m^2 per person helps ensure the restaurant keeps a safe density and prevents issues like crowd crushes. At the micro level, the Corrective Movement Advisor makes sure that robots do not just come to an emergency stop when they detect casual contact. Instead, our system actively calculates and carries out exact retreat distances to safely handle spatial conflicts while keeping the restaurant floor running smoothly.

Completing this coursework offered valuable experience in managing complex object-oriented relationships, especially through inheritance (public class StaticFoodServing extends DynamicFoodServing) and encapsulation. Dealing with

complex integration issues and boundary cases, such as making sure usable dining areas always have at least one person, required a lot of research, discussion, and careful troubleshooting with our group. This teamwork highlighted the importance of solid design in safety-focused applications. In the end, the FSMS serves as a professional-grade prototype, showing how innovative, sensor-ready Java systems can be used to improve public safety and operational efficiency.